- DI helps you implement inversion of control, a technique that promotes loose coupling of code.
- Services are injected into classes that depend on them as explicit dependencies.
- The dependency inversion principle (DIP) states that high-level classes and low-level classes should depend on abstractions, such as interfaces.
- You configure services in Program.cs via the `WebApplication Services` property, which represents the configured services for the application.
- Services are registered as a type and an implementation with a service container.
- Services are registered using one of three lifetimes: singleton, transient, or scoped.
- Only one instance of a singleton can exist. It lasts for the lifetime of the container.
- Transient services are resolved every time they are requested.
- Scoped services last for the duration of a web request in ASP.NET Core.
- Multiple implementations of the same service can be registered. The last one registered will be resolved.
- You can access all registered implementations by injection and `IEnumerable <ServiceType>`.
- You can inject into page handler methods by prefixing the service parameter with `[FromServices]`.
- You can inject directly into Razor pages via the `@inject` attribute.